

Guidelines for preparing the GSoC 2026 application proposal

Updated March 2, 2026

The purpose of this document is to provide additional information for contributors on how to prepare their proposal for a successful application under the **Google Summer of Code 2026** with the organization **Learning Equality**.

1. Read carefully everything in the [GSoC Contributor Guide](#), and especially the [Writing a proposal](#) section, as it already contains all the important information and recommendations necessary to prepare your application proposal.
2. Browse the Kolibri [code repository](#) on GitHub, check the opened and [closed issues](#), and [pull requests](#) (PRs) that fixed them. This will help you get to know the team and our development workflows in action.

If you want to contribute PRs to fix some of the issues, please do, but remember that the number of resolved issues is not the most important selection criteria.

Please ask to be assigned ONLY for issues that have the 'help wanted' label!

3. Read the [general guidelines for contributing to Kolibri](#). These are open to everyone, so not only GSoC related, but still offer the best advice on how to start contributing.
4. If you find some of our [project ideas](#) interesting, **read carefully the [Kolibri developer documentation](#) before you start asking questions in Slack**. The [Getting started](#) section of the dev docs will help you set up Kolibri for development on your own device.

Important!

We recommend you use **macOS or Linux** devices for setting up the dev environment. If you only have access to a Windows device, you should use [WSL](#) or install [VirtualBox](#) and create an Ubuntu virtual machine for Kolibri development.

Note about the use of Large Language Models

DO NOT use a simple **LLM's (ChatGPT, Claude, Gemini or similar) output as the sole attempt at any type of submission** (question on Slack, reply to a mentor, PR, or much less the application proposal text), without a prior very specific, thoroughly researched, and carefully worded prompting. Keep in mind that it is **very easy to recognize** whenever someone just feeds a LLM with the text copied from any of our project ideas, and presents that same output

as their own application proposal. **Doing this will significantly DECREASE your possibilities at accepted proposal outcome.**

Remember!

Demonstrating the ability to do independent research, and being proactive in looking for answers will benefit your application much more than generic questions like *Help me get started!*

5. Follow the instructions in the [Kolibri user guide](#) to create different users, classes, groups, and import some content. Play around with creating lessons and quizzes, then log in as a **Learner**, and score some points by doing exercises or viewing other types of content. Log in as a **Coach** and review how your progress as a learner is presented to the coach.
6. **Work on the [project-specific tasks](#).**
7. You can also visit our [Community Forums](#) to gain insights into first hand-experiences of our users and challenges they face, both structural and during the implementation of Kolibri in their local communities. This will also help orient your work, and direct you toward real world problems that one of your projects may contribute to solve.
8. Once you've selected a project idea, start working on your proposal. Review the suggested [Elements of a Quality Proposal](#) from the GSoC guide as a structure, but when you are ready to start writing, make a copy of this [Proposal Template for the Project Idea](#) document.

DO NOT present a proposal if you have not interacted with mentors on Slack. Prior exchange of ideas about projects with our team members is paramount for a successful participation in this project, and **proposals from people that have not interacted with us on Slack will NOT be taken in consideration.**

Examples of successful proposals from past editions: [Proposal 1](#), [Proposal 2](#), [Proposal 3](#)

9. Mentors will be available to review the application proposal while you are working on it. When you have advanced sufficiently for a review, you can share your application proposal draft privately with mentors to receive feedback.
10. Keep in mind the official [GSoC timeline](#) while working on your application proposal!

To maintain the exchange of information centralized, please post comments and questions in the **#gsoc-2026** Slack channel.

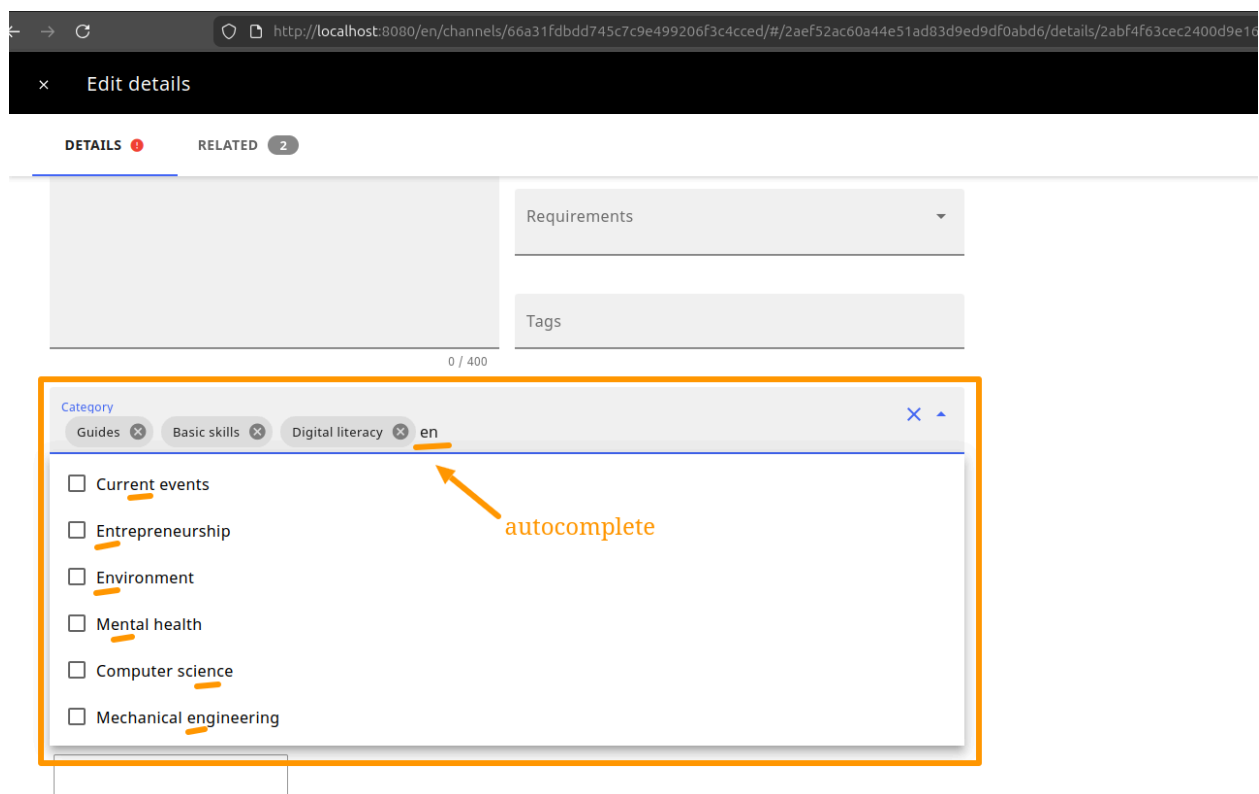
Happy coding!

Learning Equality GSoC Team

Project-specific tasks

Replace Vuetify Multi-Select Component In Kolibri Studio “New Exercise” Page

The purpose of this project is to introduce a new multi-select component into our ecosystem via Kolibri Studio. Your proposal should demonstrate strong familiarity with Kolibri Studio and the Kolibri Design System, showing you have done substantive investigation of our ecosystem. Generic proposals that demonstrate only a basic level of engagement with our specific ecosystem products will not be considered. The project consists of two main tasks. One, develop an initial accessible public API for a component that can be used to replace Vuetify’s `VAutocomplete`. Two, build that component and use it in place of the `CategoryOptions` dropdown within Studio’s “New Exercise” page.



This component should lay the foundation for a future design system component. *Implementing it in KDS and replacing all instances in our ecosystem is out of scope for this project.*

Suggested resources to familiarize yourself with Studio:

- Checkout the [Kolibri Studio repo](#) and [documentation](#)
- Set up Studio to run locally, and get familiar with the project structure, basic workflows like creating a channel, importing or uploading resources, and editing

resources like questions.

Strong proposals will demonstrate the candidate has:

- Studied the Vuetify VCombobox and VAutocomplete API documentation.
- Studied [KDS documentation](#) and learned about existing KDS form components (KSelect, KTextbox, KButton).
- Explored our approach to internationalization, including translation and RTL language support.
- Considered what a longer term, scalable public API could look like, and what is a realistic initial implementation that is achievable during the GSoC period.

In addition to the general template format, proposals for this project should include:

- **Relevant accessibility research**, general strategy, and a proposal for an accessible implementation.
- **Component architecture plan** including component API (props, events, slots) *that will support all current UX needs in Studio*, state management approach, and performance considerations. Explain how your proposed API sets the foundation for a further stable and scalable architecture
- **Rationale for all decisions**, pros-and-cons, potential limitations, and other options that were considered.
- **A comprehensive testing strategy**. What types of tests or testing are necessary for high quality work? When do they need to happen?
- **Discussion of risks and uncertainties**. What challenges do you anticipate and what support or resources you may need for your most successful project?

Introduce user-generated UI themes in Kolibri to support accessibility and customization

The purpose of this project is to create UI for device-wide and user-level theme customization in Kolibri, enabling branding and accessibility features. Your proposal should demonstrate strong familiarity with Kolibri's architecture, theme system, and existing patterns, showing you understand how themes currently work and how settings are managed across the platform.

Suggestions to familiarize yourself with Kolibri:

- Checkout [the Kolibri repo](#) and documentation at [kolibri-dev.readthedocs.io](#)
- Set up Kolibri to run locally, and navigate through Device settings and User profile/settings
- Study `packages/kolibri/styles/internal/themeSpec.js` and trace how theme tokens are used throughout the codebase
- Explore the plugin architecture and how styles are loaded across plugins (Learn, Coach, Device, Facility)

Strong proposals will demonstrate the candidate has:

- Studied how `$themeTokens`, `$themeBrand`, and `$themePalette` work and how Vue components access them via computed styles
- Reviewed KDS Colors and Styling documentation at [design-system.learningequality.org](#)
- Investigated existing models (FacilitySettings, DeviceSettings, UserProfile) and how settings are persisted and loaded
- Explored existing settings UI patterns, form validation, and how settings changes are applied
- Researched WCAG 2.1 Level AA contrast requirements and tools for checking contrast ratios
- Tested changing theme values in `themeSpec.js` locally to see how changes propagate

In addition to the general template format, proposals for this project should include:

- **Plan for theme specifications** with token mappings for Default Kolibri, High Contrast (WCAG AAA), and Dark Mode, including contrast ratio calculations and approaches for text size (+125%) and line spacing (+150%) scaling
 - These token mappings should be pre-configured in the code, and function as an overlay on top of the theme, rather than something the user can manually refine and set themselves
- **Database architecture proposal** including which models to extend, field names/types, migration strategy, and how user overrides will interact with device defaults
 - The proposal should introduce a Django model, `'Theme'`, which has a field for each setting, and should map to the existing API for our default theme plugin
 - Our current "default theme" should be read from the database as part of these changes

- **Backend API design** with endpoint specifications, request/response formats, permission checking, and error handling strategy
- **Frontend architecture plan** for how theme tokens will be initialized from database, how changes will propagate reactively to all components and across browser tabs/windows, and preview functionality implementation
- **Comprehensive testing strategy** covering database migrations, API permissions, theme token propagation across plugins, accessibility validation for all theme variants, and cross-browser/device testing
- **Discussion of risks including challenges** with theme propagation, performance concerns, browser compatibility, and migration risks for existing installations
- **Note on UI Design:** The final UI spec will be provided by Learning Equality design staff during the project. Please focus your proposal on architectural plans, API design, and state management rather than specific layouts.

Update EPub rendering to a safely sandboxed viewer, using the maintained foliate-js library

The purpose of this project is to replace Kolibri's current EPub.js library with the actively maintained [foliate-js](#) library, implementing it within a sandboxed viewer for security. Your proposal should demonstrate understanding of both the `foliate-js` library and Kolibri's sandboxing architecture, as well as a commitment to maintain and improve the existing accessibility features.

Suggestions to familiarize yourself with Kolibri:

- Checkout the Kolibri repo and documentation at kolibri-dev.readthedocs.io
- Set up Kolibri to run locally and locate the existing `epub_viewer` plugin code
- Test the current EPUB viewer with various EPUB files to understand existing functionality and identify bugs. You can use our Kolibri QA Channel to access EPUB files using the channel token 'nakav-mafak'
- Review Kolibri's browserlist configuration and understand browser compatibility requirements
- Explore the kolibri-sandbox architecture and how it's used to safely render potentially unsafe HTML5, H5P, and Bloompublish content

Strong proposals will demonstrate the candidate has:

- Researched the `foliate-js` library documentation and examples, understanding its features, API, and EPUB3 SMIL support
- Analyzed `foliate-js` JavaScript requirements against Kolibri's browserlist configuration to identify any polyfill/ponyfill needs
- Studied the kolibri-sandbox plugin architecture, including how it isolates JavaScript execution and communicates across the `postMessage` boundary with the parent application
- Investigated current `epub_viewer` plugin functionality including navigation, bookmarking, progress tracking, and accessibility features
- Reviewed WCAG accessibility standards and EPUB3 SMIL specifications for read-aloud and synchronized text-audio features
- Explored Kolibri's UI patterns and design system to understand how to align the viewer interface with platform standards

In addition to the general template format, proposals for this project should include:

- **Compatibility analysis of `foliate-js`** with Kolibri's browserlist, identifying any JavaScript features requiring polyfills/ponyfills, potential browser compatibility issues, and proposed solutions
- **Sandboxing architecture plan** covering how `foliate-js` will be integrated into kolibri-sandbox, communication strategy between sandbox and parent application, security considerations for untrusted EPub content, and how existing viewer functionality will be preserved
- **Accessibility strategy** including how to retain current accessibility features, plan for implementing EPUB3 SMIL support, WCAG compliance verification approach, and keyboard navigation/screen reader testing

- **Migration and testing plan for handling existing EPub functionality**, comparing `foliate-js` rendering against current EPub.js behavior, validating progress tracking and bookmarking work correctly, and comprehensive testing with various EPub formats and edge cases
- Comprehensive testing strategy covering security testing of sandboxed rendering, accessibility testing with screen readers, cross-browser compatibility testing, performance testing with large EPub files, and validation that known bugs are resolved
- **Discussion of risks** including challenges with `foliate-js` integration, sandboxing limitations or constraints, maintaining feature parity during migration, potential performance issues, and what support or resources may be needed
- **Note on UI Design:** The final UI spec will be provided by Learning Equality design staff during the project. Please focus your proposal on architectural plans, API design, and state management rather than specific layouts.

Learner-Facing QTI Interactions

The purpose of this project is to expand Kolibri's QTI 3.0 rendering capabilities to support additional interaction types beyond the current Multiple Choice and Text Entry implementations. Your proposal should demonstrate understanding of the QTI 3.0 specification, Kolibri's existing QTI viewer plugin, and the architectural considerations for rendering interactive assessments across devices.

Suggestions to familiarize yourself with Kolibri:

- Checkout the Kolibri repo and documentation at kolibri-dev.readthedocs.io
- Set up Kolibri to run locally and locate the QTI viewer plugin code
- Study how Multiple Choice and Text Entry are currently rendered and test the existing QTI interactions
- Understand the composable structure used in the QTI viewer plugin, and how additional interaction types can be added and leverage the data flow
- Understand how the SafeHTML viewer component is used with custom component to render QTI interactions inline with other HTML content in a way that prevents XSS and other injection attacks
- Explore how exercises, practice quizzes, and coach-created quizzes work in Kolibri

Strong proposals will demonstrate the candidate has:

- Studied the QTI 3.0 specification at imglobal.org, focusing on the XML structure, response processing, and grading rules for QTI
- Demonstrate an understanding of the schema of inline choice, match interaction, order interaction, choice interaction (true/false), text entry (fill-in-the-blank), and gap match (drag-and-drop)
- Researched mobile responsiveness, RTL language support, and accessibility standards (WCAG 2.1 Level AA) requirements for Kolibri
- Explored accessible interaction patterns, particularly for drag-and-drop with keyboard alternatives
- Investigated performance considerations for low-end devices and cross-browser compatibility
- Understood how QTI assessments will integrate with existing assessment workflows (exercises, quizzes)

In addition to the general template format, proposals for this project should include:

- **QTI specification analysis for each priority interaction type** covering XML structure, response processing/grading logic, accessibility considerations, mobile interaction patterns, and RTL support requirements
- **Architecture plan** for extending the existing QTI viewer plugin, including component structure, and shared utilities - whether any updates are needed for the XML parsing strategy, and response validation/grading
- **Accessibility strategy aiming for WCAG 2.1 Level AA compliance** approach for each interaction type, keyboard navigation patterns (especially for drag-and-drop), screen reader implementation, and alternative interaction methods where needed
- **Mobile and RTL considerations** including touch interaction patterns, responsive layouts, and testing approach for various devices

- **Implementation priority recommendation** with ordering rationale (easiest to hardest), dependencies between types, and reusable components across types
- **Comprehensive testing strategy**
- **Discussion of risks** including challenges with specific interaction types and backward compatibility concerns
- **Note on UI Design:** The final UI spec will be provided by Learning Equality design staff during the project. Please focus your proposal on architectural plans, API design, and state management rather than specific layouts.

Editing of QTI question types

The purpose of this project is to create authoring tools in Kolibri Studio for QTI 3.0 questions, enabling content creators to build assessments directly in Studio rather than importing pre-existing QTI packages. Your proposal should demonstrate understanding of QTI 3.0 XML generation, Studio's content pipeline, and the authoring experience needed for creating accessible assessments.

Suggestions to familiarize yourself with Studio:

- Checkout the Kolibri Studio repo and documentation
- Set up Studio to run locally and understand content creation and editing workflows
- Study how resources are created, edited, packaged, and transferred to Kolibri
- Review how current exercise/question editing works, including form patterns, validation, and error handling
- Test how Multiple Choice and Text Entry questions are currently created and identify reusable patterns
- Review how Pydantic models are used to validate QTI XML, HTML, and MathML for a limited subset of interactions

Strong proposals will demonstrate the candidate has:

- Studied the QTI 3.0 specification at [imsglobal.org](https://www.imsglobal.org/), focusing on XML structure for: inline choice, match interaction, order interaction, choice interaction (true/false), text entry (fill-in-the-blank), and gap match (drag-and-drop)
- Investigated response processing and grading configuration, and how correct answers and scoring rules are defined in QTI XML
- Understood how QTI packages are structured and how Kolibri's QTI viewer plugin consumes QTI files
- Reviewed Studio's existing form components, Vue.js form handling patterns (Vue 2.x), and validation approaches
- Explored the content database schema, models, and how content is packaged for transfer to Kolibri
- Tested importing and viewing QTI content in Kolibri to understand expected format

In addition to the general template format, proposals for this project should include:

- **QTI XML generation strategy** covering, for each priority interaction type: required authoring UI inputs, XML structure to be generated, response processing configuration, validation rules, and example XML output; plus approach for ensuring valid QTI 3.0 and testing compatibility with Kolibri viewer
- **Frontend architecture** covering how QTI XML and intermixed rich text content are manipulated on the frontend, giving rationale for either editing of XML/HTML directly, or an intermediary format that gets handled in the backend
- **Backend architecture** covering database schema for storing QTI question data, API endpoints for CRUD operations, XML generation logic location and testing approach, and integration with Studio's content pipeline
- **Implementation priority recommendation** with ordering rationale (complexity, dependencies, reusability), which types share authoring patterns, and which require unique UI approaches

- **Comprehensive testing strategy**
- **Discussion of risks** including challenges with complex QTI features, handling unsupported viewer features, validation strategies for catching authoring errors early, version update considerations, and testing support needs
- **Note on UI Design:** The final UI spec will be provided by Learning Equality design staff during the project. Please focus your proposal on architectural plans, API design, and state management rather than specific layouts.